

# Gplearn的各种使用：

## gplearn库中的Symbolic Transformer

Symbolic Transformer是一种有监督的转换器，它首先通过构建一组随机公式来表示关系。这些公式被表示为树状结构，数学函数递归地应用于变量和常量。每一代程序都是从前一代进化而来的，通过选择种群中最适合的个体进行遗传操作，如交叉、变异或复制。最终的种群中搜索出彼此之间相关性最小的最适合个体。

### 参数介绍

gplearn库中的Symbolic Transformer是一种基于遗传编程的特征工程工具。它通过进化算法生成新的特征，以提高机器学习模型的性能。以下是一些关键参数的介绍：

| 参数名称               | 定义和参数设置说明                                                                                                                         | 参数设置         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------|
| generations        | 公式进化的世代数量。世代数量越多，消耗算力越多，公式的进化次数越多。                                                                                                | 3            |
| population_size    | 每一代公式群体中的公式数量。公式数量越大，消耗算力越多，公式之间组合的空间越大。                                                                                          | 1000         |
| function_set       | 用于构建和进化公式时使用的函数集，可自定义更多函数。                                                                                                        | 使用图表8中的函数集   |
| init_depth         | 公式树的初始化深度，init_depth是一个二元组(min_depth, max_depth)，树的初始深度将处在[min_depth, max_depth]区间内。设置树深度最小1层，最大4层。最大深度越深，可能得出越复杂的因子，但是因子的意义更难解释。 | (1,4)        |
| tournament_size    | 每一代的所有公式中，tournament_size个公式会被随机选中，其中适应度最高的公式能进行变异或繁殖生成下一代公式。tournament_size越小，随机选择范围越小，选择的结果越不确定                                 | 20           |
| metric             | 适应度指标，可自定义更多指标。                                                                                                                   | 自定义的RankIC指标 |
| p_crossover        | 父代进行交叉变异进化的概率。交叉变异是最有效的进化方式，可以设置为较大概率。                                                                                            | 0.4          |
| p_subtree_mutation | 父代进行子树变异进化的概率。子树变异的结果不稳定，概率不宜过大。                                                                                                  | 0.01         |

| 参数名称                  | 定义和参数设置说明                                                                                                                                                                                                                                                                                                                     | 参数设置  |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| p_hoist_mutation      | 父代进行Hoist变异进化的概率。<br>本文的测试中公式树层次都较低，<br>所以没有使用Hoist变异。                                                                                                                                                                                                                                                                        | 0     |
| p_point_mutation      | 父代进行点变异进化的概率。点变异的结果不稳定，<br>概率不宜过大。                                                                                                                                                                                                                                                                                            | 0.01  |
| p_point_replace       | 即点变异中父代每个节点进行变异进化的概率。<br>点变异的概率已经很小，<br>可设置为较大概率保证点变异的执行。                                                                                                                                                                                                                                                                     |       |
| stopping_criteria     | 提前停止进化所需的度量值。                                                                                                                                                                                                                                                                                                                 |       |
| n_jobs                | 拟合并行运行的作业数。如果为-1，<br>则作业数设置为内核数。                                                                                                                                                                                                                                                                                              | -1    |
| verbose               | 控制进化构建过程的冗长程度。                                                                                                                                                                                                                                                                                                                | 1     |
| hall_of_fame          | 在为 n_components<br>寻找相关性最小的个体时进行比较的最合适程序的数量。<br>如果为“无”，则将使用整个最终生成结果。                                                                                                                                                                                                                                                          | 100   |
| n_components          | 在名人堂（hall_of_fame）中搜索相关性最小的个体后，<br>返回的最佳节目数量。如果为“无”，将使用整个名人堂。                                                                                                                                                                                                                                                                 |       |
| parsimony_coefficient | 这个常数通过调整程序的适应度，使其更不利于选择，<br>从而对大型程序进行惩罚。数值越大，<br>对程序的惩罚就越大，这可以控制所谓的“膨胀”现象。<br>臃肿是指进化过程中程序的规模不断扩大，<br>而适配度却没有显著提高，这不仅耗费计算时间，<br>而且最终结果也不容易理解。<br>这一参数可能需要在连续运行时进行调整。(如果使用<br>“auto”（自动），则每一代的解析系数都会使用 $c = \text{Cov}(l,f)/\text{Var}(l)$ 重新计算，其中 $\text{Cov}(l,f)$ 是种群中程序大小 $l$ 与程序适合度 $f$ 之间的协方差， $\text{Var}(l)$ 是程序大小的方差。) | 0.005 |
| max_samples           | 对每个程序进行评估时，从 X 中抽取样本的比例。                                                                                                                                                                                                                                                                                                      | 0.9   |

## 例子

```
function_set = ['add', 'sub', 'mul', 'div',
               'sqrt', 'log', 'abs', 'neg', 'inv',
               'max', 'min']

gp = SymbolicTransformer(generations=20, population_size=2000,
                        hall_of_fame=100, n_components=10,
                        function_set=function_set,
                        #parsimony_coefficient=0.0005,
                        max_samples=0.9, verbose=1,
                        random_state=0, n_jobs=-1, metric = 'spearman')

result = gp.fit(df, target)

## 或者:
# result_fit_transform = gp.fit_transform(diabetes.data[:300, :], diabetes.target[:300])
# 拟合完数据后直接进行转换: 返回 nrow * n_components 的数据
```

结果:

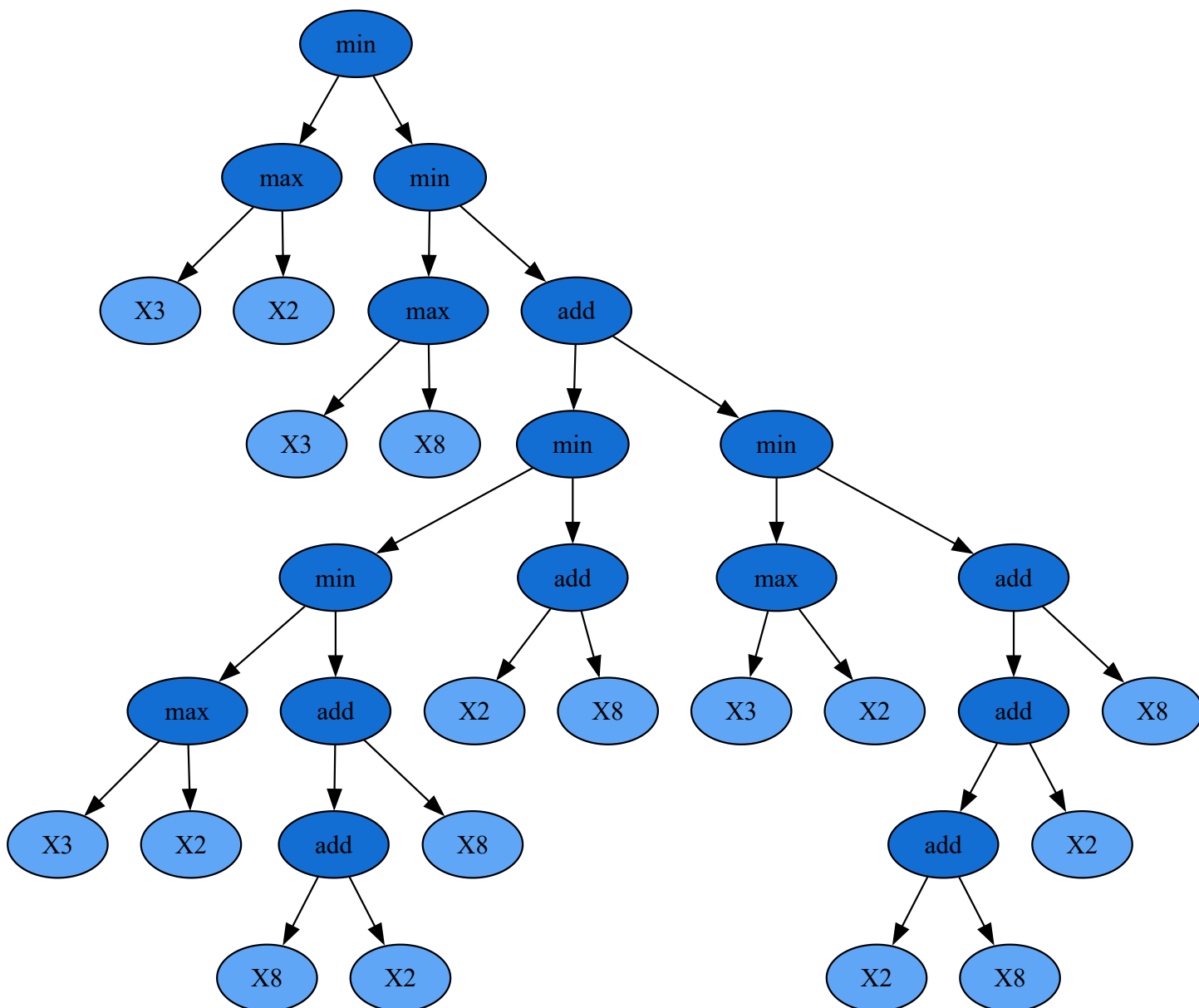
| Population Average |        |          | Best Individual |          |             |           |
|--------------------|--------|----------|-----------------|----------|-------------|-----------|
| Gen                | Length | Fitness  | Length          | Fitness  | OOB Fitness | Time Left |
| 0                  | 11.37  | 0.140571 | 7               | 0.60953  | 0.524855    | 1.20m     |
| 1                  | 7.15   | 0.35238  | 3               | 0.659272 | 0.309861    | 51.57s    |
| 2                  | 5.44   | 0.466162 | 4               | 0.66782  | 0.288364    | 57.89s    |
| 3                  | 4.49   | 0.586366 | 3               | 0.671457 | 0.320572    | 46.97s    |
| 4                  | 3.99   | 0.597676 | 7               | 0.688368 | 0.161787    | 41.72s    |
| 5                  | 3.65   | 0.605824 | 7               | 0.684906 | 0.135113    | 39.34s    |
| 6                  | 3.58   | 0.611524 | 4               | 0.680278 | 0.29176     | 37.19s    |
| 7                  | 3.68   | 0.612427 | 3               | 0.674306 | 0.339035    | 33.66s    |
| 8                  | 4.08   | 0.611616 | 7               | 0.677397 | 0.32696     | 30.98s    |
| 9                  | 4.78   | 0.610042 | 7               | 0.680798 | 0.323262    | 33.81s    |
| 10                 | 5.51   | 0.611407 | 9               | 0.68331  | 0.320245    | 25.56s    |
| 11                 | 6.44   | 0.613251 | 9               | 0.678225 | 0.466664    | 23.09s    |
| 12                 | 7.15   | 0.616251 | 9               | 0.700843 | 0.0830612   | 20.81s    |
| 13                 | 8.12   | 0.617384 | 7               | 0.68119  | 0.344517    | 17.17s    |
| 14                 | 8.96   | 0.618892 | 13              | 0.684455 | 0.311955    | 14.51s    |
| 15                 | 9.51   | 0.620367 | 19              | 0.690903 | 0.205063    | 12.06s    |
| 16                 | 9.68   | 0.619547 | 11              | 0.689026 | 0.449675    | 8.84s     |
| 17                 | 9.89   | 0.621826 | 13              | 0.691072 | 0.386541    | 5.84s     |
| 18                 | 10.87  | 0.625144 | 29              | 0.695689 | 0.385131    | 2.86s     |
| 19                 | 12.40  | 0.627153 | 11              | 0.702239 | 0.409255    | 0.00s     |

CPU times: user 20.3 s, sys: 1.33 s, total: 21.6 s

Wall time: 59.8 s

可以看到，对一个300行 \* 10列的数据需要消耗 1 min

```
import graphviz
dot_data = gp._best_programs[-1].export_graphviz()
graph = graphviz.Source(dot_data)
graph
```



```
gp.get_params() ## 获得gp的超参数设置
gp_features = gp.transform(df)
new_diabetes = np.hstack((df, gp_features))
```

**针对时间序列的自定义函数，针对于二维 时间 × 特征 的情况，三维数据 时间 × 特征 × 股票 的讨论在下一部分**

```
import numpy as np
import numba
from numba import jit
```

下面常用算子作用分别是：

- `_delta_x`: 计算一阶差分

- `_delay_x`: x阶延迟lag

# 自定义函数, make\_function函数群

```
def _delta_1(data):
    value = np.diff(data.flatten())
    value = np.append(0, value)
    return value

def _delta_2(data):
    value = np.diff(data.flatten(),2)
    value = np.append([0,0], value)
    return value

def _delta_3(data):
    value = np.diff(data.flatten(),3)
    value = np.append([0,0,0], value)
    return value

def _delay_1(data):
    period = 1
    value = pd.Series(data.flatten()).shift(period)
    value = np.nan_to_num(value)
    return value

def _delay_2(data):
    period = 2
    value = pd.Series(data.flatten()).shift(period)
    value = np.nan_to_num(value)
    return value

def _delay_3(data):
    period = 3
    value = pd.Series(data.flatten()).shift(period)
    value = np.nan_to_num(value)
    return value
```

下面常用算子作用分别是：

- `_delta`: 计算一阶差分
- `_delay`: 用data[i-2]的数据代替data[i]
- `_ts_sum_numba`: 计算滚动和
- `_ts_mean_numba`: 计算滚动均值
- `_ts_std_numba`: 计算滚动标准差
- `_ts_rank_numba`: 计算滚动排名
- `_ts_prod_numba`: 计算滚动乘积
- `_ts_min_numba`: 计算滚动最小值

- `_ts_max_numba`: 计算滚动最大值
- `_ts_scale_numba`: 计算滚动z-score
- `_ts_argmax_numba`: 计算滚动最大值位置
- `_ts_argmin_numba`: 计算滚动最小值位置
- `_ts_beta`: 计算滚动回归
- `_ts_intercept`: 计算滚动截距
- `_ts_residuals`: 计算滚动残差
- `_ts_pearson`: 计算滚动皮尔逊相关系数
- `_ts_spearman`: 计算滚动斯皮尔曼相关系数
- `_ts_covariance`: 计算滚动协方差

```
@numba.njit( numba.float64[:] (numba.float64[:], numba.int64), cache=True)
```

```
def rolling_sum_numba(data, n):
    res = np.empty(len(data), dtype=np.float64)
    # 缓存的窗口内数据求和结果
    nobuffer_flag = True
    sum_buffer = 0
    res[0] = data[0]
    for i, x in enumerate(data[1:n], 1):
        res[i] = res[i-1] + x
    for i, x in enumerate(data[n:], n):
        head_x = data[i-n]
        # 如果缓存结果为空, 则先通过遍历求第一次结果
        if nobuffer_flag:
            nobuffer_flag = False
            sum_tick = 0
            for tick in data[1:n+1]:
                sum_tick += tick
            res[i] = sum_tick
            # 将求和结果缓存下来
            sum_buffer = sum_tick
        else:
            # 这里的算法将计算复杂度从O(n)降低到了O(1)
            sum_buffer = sum_buffer - head_x + x
            res[i] = sum_buffer
    return res
```

```
# 左开右闭
```

```
def _ts_sum_numba(data):
    window = 10
    value = rolling_sum_numba(data.astype(np.float64), window)
    value = np.nan_to_num(value)
    return value
```

```
@numba.njit( numba.float64[:] (numba.float64[:], numba.int64), cache=True)
```

```
def rolling_mean_numba(data, n):
    res = np.empty(len(data), dtype=np.float64)
    nobuffer_flag = True
    sum_buffer = 0
    res[0] = data[0]
    for i, x in enumerate(data[1:n], 1):
        res[i] = (res[i-1]*i+x)/(i+1)
    for i, x in enumerate(data[n:], n):
        head_x = data[i-n]
        if nobuffer_flag:
            nobuffer_flag = False
            sum_tick = 0
            for tick in data[1:n+1]:
```



```

        sum_tick += tick
        res[i] = sum_tick/n
        sum_buffer = sum_tick
    else:
        sum_buffer = sum_buffer - head_x + x
        res[i] = sum_buffer/n
    return res

# 左开右闭
def _ts_mean_numba(data):
    window = 10
    value = rolling_mean_numba(data.astype(np.float64), window)
    value = np.nan_to_num(value)
    return value

@jit(numba.float64[:](numba.float64[:], numba.int64), cache=True)
def rolling_std_numba(data, n):
    res = np.empty(len(data), dtype=np.float64)
    # 保护标准差 (如果为0, 标准差变为0.001)
    res[0] = 0.001
    for i, x in enumerate(data[1:n], 1):
        res[i] = np.std(data[:i+1])
    for i, x in enumerate(data[n:], n):
        res[i] = np.std(data[i-n+1:i+1])
    return res

# 左开右闭
def _ts_std_numba(data):
    window = 10
    value = rolling_std_numba(data.astype(np.float64), window)
    value = np.where(np.abs(value) > 0.001, value, 0.001)
    value = np.nan_to_num(value)
    return value

@numba.njit(numba.float64[:](numba.float64[:], numba.int64), cache=True)
def rolling_rank_numba(data, n):
    res = np.empty(len(data), dtype=np.float64)
    res[0] = 1
    for i, x in enumerate(data[1:n], 1):
        idx = np.argsort(data[:i+1])
        res[i] = np.argsort(idx)[-1] + 1
    for i, x in enumerate(data[n:], n):
        idx = np.argsort(data[i-n+1:i+1])
        res[i] = np.argsort(idx)[-1] + 1
    return res

def _ts_rank_numba(data):
    window = 10

```

```

value = rolling_rank_numba(data.astype(np.float64), window)
value = np.nan_to_num(value)
return value

@numba.njit(numba.float64[:](numba.float64[:], numba.int64), cache=True)
def rolling_prod_numba(data, n):
    res = np.empty(len(data), dtype=np.float64)
    nobuffer_flag = True
    prod_buffer = 1
    res[0] = data[0]
    for i, x in enumerate(data[1:n], 1):
        res[i] = res[i-1] * x
    for i, x in enumerate(data[n:], n):
        head_x = data[i-n]
        # 如果缓存结果为空, 则先通过遍历求第一次结果
        if nobuffer_flag:
            nobuffer_flag = False
            prod_tick = 1
            for tick in data[1:n+1]:
                prod_tick *= tick
            res[i] = prod_tick
            # 将乘积结果缓存下来
            prod_buffer = prod_tick
        else:
            if head_x == 0: # pop出的元素为0
                prod_buffer = 1
                for j, y in enumerate(data[i-n+1:i+1], i-n+1):
                    prod_buffer *= y
            else: # pop出的元素不为0
                prod_buffer /= head_x
                prod_buffer *= x
            res[i] = prod_buffer
    return res

# 左开右闭
def _ts_prod_numba(data):
    window = 10
    value = rolling_prod_numba(data.astype(np.float64), window)
    value = np.nan_to_num(value)
    return value

@numba.njit(numba.float64[:](numba.float64[:], numba.int64), cache=True)
def rolling_min_numba(data, n):
    res = np.empty(len(data), dtype=np.float64)
    res[0] = data[0]
    for i, x in enumerate(data[1:n], 1):
        res[i] = x if x < res[i-1] else res[i-1]
    for i, x in enumerate(data[n:], n):

```

```

        res[i] = min(data[i-n+1:i+1])
    return res

# 左开右闭
# 可以像_ts_rank那样用单调队列或者最小堆，但似乎没有这个快
def _ts_min_numba(data):
    window = 10
    value = rolling_min_numba(data.astype(np.float64), window)
    value = np.nan_to_num(value)
    return value

@numba.njit(numba.float64[:] (numba.float64[:], numba.int64), cache=True)
def rolling_max_numba(data, n):
    res = np.empty(len(data), dtype=np.float64)
    res[0] = data[0]
    for i, x in enumerate(data[1:n], 1):
        res[i] = x if x > res[i-1] else res[i-1]
    for i, x in enumerate(data[n:], n):
        res[i] = max(data[i-n+1:i+1])
    return res

# 左开右闭
def _ts_max_numba(data):
    window = 10
    value = rolling_max_numba(data.astype(np.float64), window)
    value = np.nan_to_num(value)
    return value

def _ts_scale_numba(data):
    value_mean = _ts_mean_numba(data)
    value_std = _ts_std_numba(data)
    value = (data - value_mean) / value_std
    value = np.where(value > 5, 5, value)
    value = np.where(value < -5, -5, value)
    value = np.nan_to_num(value)
    return value

# 复杂度O(data.shape[0] + window)
@numba.njit(numba.int64[:] (numba.float64[:], numba.int64), cache=True)
def rolling_argmax_numba(data, n):
    """
    leetcode239 单调队列
    复杂度 O(N + n), N为样本数量

    :param data:
    :param n: 回溯窗口大小
    :return:
    """

```

```

res = np.empty(len(data), dtype=np.int64)
head = 0
tail = 0
que_idx = np.empty(len(data), dtype=np.int64)
for i, x in enumerate(data[:n]):
    while tail > 0 and data[que_idx[tail - 1]] < x:
        tail -= 1
    que_idx[tail] = i
    tail += 1
    res[i] = que_idx[0] + 1
for i, x in enumerate(data[n:], n):
    if que_idx[head] <= i - n:
        head += 1
    while tail > head and data[que_idx[tail - 1]] < x:
        tail -= 1
    que_idx[tail] = i
    tail += 1
    res[i] = que_idx[head] - i + n
return res

def _ts_argmax_numba(data):
    window = 10
    value = rolling_argmax_numba(data.astype(np.float64), window)
    value = np.nan_to_num(value)
    return value

# 复杂度O(data.shape[0] + window)
@numba.njit(numba.int64[:](numba.float64[:,], numba.int64), cache=True)
def rolling_argmin_numba(data, n):
    res = np.empty(len(data), dtype=np.int64)
    head = 0
    tail = 0
    que_idx = np.empty(len(data), dtype=np.int64)
    for i, x in enumerate(data[:n]):
        while tail > 0 and data[que_idx[tail - 1]] > x:
            tail -= 1
        que_idx[tail] = i
        tail += 1
        res[i] = que_idx[0] + 1
    for i, x in enumerate(data[n:], n):
        if que_idx[head] <= i - n:
            head += 1
        while tail > head and data[que_idx[tail - 1]] > x:
            tail -= 1
        que_idx[tail] = i
        tail += 1
        res[i] = que_idx[head] - i + n
    return res

```

# 左开右闭

```
def _ts_argmin_numba(data):  
    window = 10  
    value = rolling_argmin_numba(data.astype(np.float64), window)  
    value = np.nan_to_num(value)  
    return value
```

```
def rolling_regression_beta(X, Y):
    N = 10
    """
    计算 X 和 Y 在滚动 N 期窗口内的回归系数、截距、残差、相关系数和协方差。

    参数:
        X (numpy.ndarray): 自变量序列
        Y (numpy.ndarray): 因变量序列
        N (int): 滚动窗口大小

    返回:
        pd.DataFrame: 包含  $\beta$ 值、截距、残差、Pearson相关系数、Spearman相关系数、协方差 的 DataFrame
    """
    # 转换为 Pandas Series
    X = pd.Series(X.flatten())
    Y = pd.Series(Y.flatten())

    # 创建结果存储的 DataFrame
    results = pd.DataFrame(index=X.index, columns=[
        'Beta'])

    # 滚动窗口计算
    for i in range(N - 1, len(X)):
        x_window = X.iloc[i - N + 1:i + 1].values.reshape(-1, 1)
        y_window = Y.iloc[i - N + 1:i + 1].values

        # 线性回归
        linreg = LinearRegression()
        linreg.fit(x_window, y_window)

        # 存储结果
        results.loc[i] = {
            'Beta': linreg.coef_[0]
        }

    # 填充 NaN 值为 0
    with pd.option_context("future.no_silent_downcasting", True):
        results = results.fillna(0).infer_objects(copy=False)
    return results.to_numpy().flatten()

def rolling_regression_intercept(X, Y):
    N = 10
    """
    计算 X 和 Y 在滚动 N 期窗口内的回归系数、截距、残差、相关系数和协方差。

    参数:
        X (numpy.ndarray): 自变量序列

```

Y (numpy.ndarray): 因变量序列

N (int): 滚动窗口大小

返回:

pd.DataFrame: 包含  $\beta$ 值、截距、残差、Pearson相关系数、Spearman相关系数、协方差 的 DataFrame

"""

# 转换为 Pandas Series

X = pd.Series(X.flatten())

Y = pd.Series(Y.flatten())

# 创建结果存储的 DataFrame

results = pd.DataFrame(index=X.index, columns=['Intercept'])

# 滚动窗口计算

for i in range(N - 1, len(X)):

    x\_window = X.iloc[i - N + 1:i + 1].values.reshape(-1, 1)

    y\_window = Y.iloc[i - N + 1:i + 1].values

    # 线性回归

    linreg = LinearRegression()

    linreg.fit(x\_window, y\_window)

    # 存储结果

```
results.loc[i] = {
    'Intercept': linreg.intercept_
}
```

# 填充 NaN 值为 0

with pd.option\_context("future.no\_silent\_downcasting", True):

    results = results.fillna(0).infer\_objects(copy=False)

return results.to\_numpy().flatten()

def rolling\_regression\_residuals(X, Y):

N = 10

"""

计算 X 和 Y 在滚动 N 期窗口内的回归系数、截距、残差、相关系数和协方差。

参数:

X (numpy.ndarray): 自变量序列

Y (numpy.ndarray): 因变量序列

N (int): 滚动窗口大小

返回:

pd.DataFrame: 包含  $\beta$ 值、截距、残差、Pearson相关系数、Spearman相关系数、协方差 的 DataFrame

"""

# 转换为 Pandas Series

X = pd.Series(X.flatten())

Y = pd.Series(Y.flatten())

```

# 创建结果存储的 DataFrame
results = pd.DataFrame(index=X.index, columns=['Residuals'])

# 滚动窗口计算
for i in range(N - 1, len(X)):
    x_window = X.iloc[i - N + 1:i + 1].values.reshape(-1, 1)
    y_window = Y.iloc[i - N + 1:i + 1].values

    # 线性回归
    linreg = LinearRegression()
    linreg.fit(x_window, y_window)
    y_pred = linreg.predict(x_window)

    # 计算残差
    residuals = y_window - y_pred

    # 存储结果
    results.loc[i] = {
        'Residuals': residuals[-1] # 最新一期的残差
    }

# 填充 NaN 值为 0
with pd.option_context("future.no_silent_downcasting", True):
    results = results.fillna(0).infer_objects(copy=False)
return results.to_numpy().flatten()

def rolling_regression_pearson(X, Y):
    N = 10
    """
    计算 X 和 Y 在滚动 N 期窗口内的回归系数、截距、残差、相关系数和协方差。

    参数:
        X (numpy.ndarray): 自变量序列
        Y (numpy.ndarray): 因变量序列
        N (int): 滚动窗口大小

    返回:
        pd.DataFrame: 包含  $\beta$ 值、截距、残差、Pearson相关系数、Spearman相关系数、协方差 的 DataFrame
    """
    # 转换为 Pandas Series
    X = pd.Series(X.flatten())
    Y = pd.Series(Y.flatten())

    # 创建结果存储的 DataFrame
    results = pd.DataFrame(index=X.index, columns=[
        'Pearson Corr'
    ])

```



```

# 滚动窗口计算
for i in range(N - 1, len(X)):
    x_window = X.iloc[i - N + 1:i + 1].values.reshape(-1, 1)
    y_window = Y.iloc[i - N + 1:i + 1].values

    # 计算 Pearson 和 Spearman 相关系数
    #pearson_corr, _ = pearsonr(x_window.flatten(), y_window)
    if len(np.unique(x_window.flatten())) == 1 or len(np.unique(y_window)) == 1:
        pearson_corr = 0 # 或者返回 np.nan
    else:
        pearson_corr, _ = pearsonr(x_window.flatten(), y_window)
    # 存储结果
    results.loc[i] = {
        'Pearson Corr': pearson_corr
    }

# 填充 NaN 值为 0
with pd.option_context("future.no_silent_downcasting", True):
    results = results.fillna(0).infer_objects(copy=False)
return results.to_numpy().flatten()

def rolling_regression_spearman(X, Y):
    N = 10
    """
    计算 X 和 Y 在滚动 N 期窗口内的回归系数、截距、残差、相关系数和协方差。

    参数:
        X (numpy.ndarray): 自变量序列
        Y (numpy.ndarray): 因变量序列
        N (int): 滚动窗口大小

    返回:
        pd.DataFrame: 包含  $\beta$ 值、截距、残差、Pearson相关系数、Spearman相关系数、协方差 的 DataFrame
    """
    # 转换为 Pandas Series
    X = pd.Series(X.flatten())
    Y = pd.Series(Y.flatten())

    # 创建结果存储的 DataFrame
    results = pd.DataFrame(index=X.index, columns=['Spearman Corr'])

    # 滚动窗口计算
    for i in range(N - 1, len(X)):
        x_window = X.iloc[i - N + 1:i + 1].values.reshape(-1, 1)
        y_window = Y.iloc[i - N + 1:i + 1].values

        # 计算 Pearson 和 Spearman 相关系数

```

```

#spearman_corr, _ = spearmanr(x_window.flatten(), y_window)
if len(np.unique(x_window.flatten())) == 1 or len(np.unique(y_window)) == 1:
    spearman_corr = 0 # 或者返回 np.nan
else:
    spearman_corr, _ = spearmanr(x_window.flatten(), y_window)
# 存储结果
results.loc[i] = {
    'Spearman Corr': spearman_corr
}

# 填充 NaN 值为 0
with pd.option_context("future.no_silent_downcasting", True):
    results = results.fillna(0).infer_objects(copy=False)
return results.to_numpy().flatten()

def rolling_regression_covariance(X, Y):
    N = 10
    """
    计算 X 和 Y 在滚动 N 期窗口内的回归系数、截距、残差、相关系数和协方差。

    参数:
        X (numpy.ndarray): 自变量序列
        Y (numpy.ndarray): 因变量序列
        N (int): 滚动窗口大小

    返回:
        pd.DataFrame: 包含  $\beta$ 值、截距、残差、Pearson相关系数、Spearman相关系数、协方差 的 DataFrame
    """
    # 转换为 Pandas Series
    X = pd.Series(X.flatten())
    Y = pd.Series(Y.flatten())

    # 创建结果存储的 DataFrame
    results = pd.DataFrame(index=X.index, columns=['Covariance'])

    # 滚动窗口计算
    for i in range(N - 1, len(X)):
        x_window = X.iloc[i - N + 1:i + 1].values.reshape(-1, 1)
        y_window = Y.iloc[i - N + 1:i + 1].values

        # 计算协方差
        #covariance = np.cov(x_window.flatten(), y_window)[0, 1]
        if len(np.unique(x_window.flatten())) == 1 or len(np.unique(y_window)) == 1:
            covariance = 0 # 如果只有一个是常数
        else:
            covariance = np.cov(x_window.flatten(), y_window)[0,1]
        # 存储结果
        results.loc[i] = {

```

```

        'Covariance': covariance
    }

# 填充 NaN 值为 0
with pd.option_context("future.no_silent_downcasting", True):
    results = results.fillna(0).infer_objects(copy=False)
# results = results.fillna(0).infer_objects(copy=False)
return results.to_numpy().flatten()

def _my_metric(y, y_pred, w):
    y = y.reshape(-1, 1)
    result = mutual_info_regression(y, y_pred, discrete_features='auto', n_neighbors=1, copy=True, random_
    return result[0]

# make_function函数群
delta_1 = make_function(function=_delta_1, name='delta', arity=1)
delta_2 = make_function(function=_delta_2, name='delta', arity=1)
delta_3 = make_function(function=_delta_3, name='delta', arity=1)
delay_1 = make_function(function=_delay_1, name='delay', arity=1)
delay_2 = make_function(function=_delay_2, name='delay', arity=1)
delay_3 = make_function(function=_delay_3, name='delay', arity=1)
ts_mean = make_function(function=_ts_mean_numba, name='ts_mean', arity=1)
ts_std = make_function(function=_ts_std_numba, name='ts_std', arity=1)
ts_prod = make_function(function=_ts_prod_numba, name='ts_prod', arity=1)
ts_rank = make_function(function=_ts_rank_numba, name='ts_rank', arity=1)
ts_min = make_function(function=_ts_min_numba, name='ts_min', arity=1)
ts_max = make_function(function=_ts_max_numba, name='ts_max', arity=1)
ts_argmax = make_function(function=_ts_argmax_numba, name='ts_argmax', arity=1)
ts_argmin = make_function(function=_ts_argmin_numba, name='ts_argmin', arity=1)
ts_sum = make_function(function=_ts_sum_numba, name='ts_sum', arity=1)
ts_scale = make_function(function=_ts_scale_numba, name='ts_scale', arity=1)
ts_beta = make_function(function=rolling_regression_beta, name='ts_beta', arity=2)
ts_intercept = make_function(function=rolling_regression_intercept, name='ts_intercept', arity=2)
ts_residuals = make_function(function=rolling_regression_residuals, name='ts_residuals', arity=2)
ts_pearson = make_function(function=rolling_regression_pearson, name='ts_pearson', arity=2)
ts_spearman = make_function(function=rolling_regression_spearman, name='ts_spearman', arity=2)
ts_covariance = make_function(function=rolling_regression_covariance, name='ts_covariance', arity=2)

## Activate The Customized Fitness Function
my_metric = make_fitness(function=_my_metric, greater_is_better=True)

```

# 针对三维数据 时间 × 特征 × 股票 的情况

## 对数据的处理：

这时我们所有的框架都要进行一点魔改

首先是数据，原始的金融数据一般是一个表格形式，记录了不同时间不同股票的信息，如：

| 股票代码      | 交易日期       | 开盘价   | 最高价   | 最低价   | 收盘价   | 成交量  | 成交额   | 收益率     |
|-----------|------------|-------|-------|-------|-------|------|-------|---------|
| SH0000001 | 2024-01-01 | 14.71 | 15.01 | 14.42 | 14.46 | 3678 | 53197 | -0.0092 |
| SH0000002 | 2024-01-01 | 14.87 | 15.17 | 14.57 | 14.81 | 2541 | 37634 | 0.0101  |
| SH0000001 | 2024-01-02 | 14.44 | 14.72 | 14.15 | 14.42 | 2317 | 33399 | -0.0034 |
| SH0000002 | 2024-01-02 | 14.66 | 14.95 | 14.37 | 14.53 | 1511 | 21959 | -0.0188 |
| SH0000001 | 2024-01-03 | 14.33 | 14.61 | 14.04 | 14.05 | 2672 | 37554 | -0.0250 |
| SH0000002 | 2024-01-03 | 14.52 | 14.81 | 14.23 | 14.53 | 1879 | 27306 | -0.0001 |

这时 我们可以考虑使用pandas库中的pivot\_table将数据转化为三维数据，即：

```
index_name = '交易日期'

columns_name = '股票代码'

df.pivot_table(index=[index_name], columns=[columns_name], sort=True, dropna=False)
```

这样我们就可以有一个 时间 × (特征 × 股票)MultiIndex,level\_1为股票代码， level\_0为时间， 的三维的数据表

接着令

```
X_0_len = len(df.index)
X_1_len = len(df.columns.levels[0])
X_2_len = len(df.columns.levels[1])
df.to_numpy(dtype=np.double).reshape((X_0_len, X_1_len, X_2_len))
```

最终得到了一个 时间 × 特征 × 股票 的 numpy 数组。

## 对原始以及自定义函数的处理：

## 对原始以及自定义适应性函数的处理：